Проект «МК-61S mini»

Команда `trap` в М61: семантика и реализация

Версия документа: 29.07.2026

1. Назначение документа

Этот документ полностью описывает команду `trap` языка сценариев М61:

- синтаксис и пользовательскую семантику;
- проверку и активацию ловушек;
- обнаружение границы команды внутри виртуальной машины МК-61;
- останов до исполнения перехваченной команды;
- сохранение и восстановление состояния калькулятора;
- запуск обработчика и возврат командой `ret`;
- одноразовый пропуск ловушки после возврата;
- работу с вложенными сценариями, метками, `wait` и ручным запуском программы;
- ограничения команд во время обработчика;
- отмену, ошибки, пределы реализации и тестовое покрытие;
- расположение соответствующего кода в проекте.

Общее описание языка находится в документе «Сценарии М61» (`MK61s-mini-M61.md`), а команды интерактивного терминала - в документе «Терминал» (`MK61s-mini-Terminal.md`).

2. Что такое `trap`

`trap` - адресная ловушка на команду программы калькулятора. Она связывает адрес программной памяти с меткой обработчика М61:

```
trap <адрес> run :<метка>
```

Когда виртуальная машина собирается исполнить команду по активному адресу:

1. уже завершённая предыдущая команда остаётся завершённой;
2. команда по адресу ловушки ещё не исполняется;
3. виртуальная машина отдаёт управление М61;
4. М61 сохраняет полный контекст калькулятора;
5. сценарий начинает исполняться с указанной метки;
6. `ret` восстанавливает контекст;
7. та же команда ровно один раз проходит без повторного срабатывания этой ловушки;
8. при следующем приходе на адрес ловушка снова срабатывает.

`trap` не является:

- заменой байта программы;
- программной точкой останова, записанной в память калькулятора;
- перехватом нажатия клавиши;
- перехватом любого одинакового кода операции независимо от адреса;
- аппаратным прерыванием посреди команды;

- командой пошагового исполнения.

Это кооперативная остановка виртуальной машины на границе команд программы. Память программы не изменяется.

3. Краткий пример

```
hin
01
trap 01 run :frame
02
03
С/П
run
print "PROGRAM STOPPED"
ret

:frame
print "BEFORE STEP 01"
stk
ret
```

Последовательность выполнения:

1. `hin` очищает программную память и переводит калькулятор в режим ввода программы.
2. В адрес `00` записывается команда `01`.
3. Строка `trap 01 run :frame` активирует ловушку адреса `01`; сама она ничего не записывает в программную память.
4. В адреса `01` и `02` записываются команды `02` и `03`, затем С/П.
5. `run` запускает программу калькулятора с адреса `00`.
6. Команда по адресу `00` исполняется.
7. Перед командой по адресу `01` виртуальная машина передаёт управление обработчику `:frame`.
8. Обработчик может читать состояние и выводить диагностику.
9. Его `ret` восстанавливает калькулятор и разрешает исполнить команду по адресу `01`.
10. После обычного останова программы M61 продолжает сценарий со строки после `run`.

4. Синтаксис

Каноническая запись:

```
trap 10 run :handler
```

Формат имеет следующие правила:

- ключевые слова `trap` и `run` пишутся строчными латинскими буквами;
- адрес задаётся десятичным целым числом без знака;
- допустимый диапазон адресов - от `0` до `111`;
- перед ключевыми словами и между элементами допускаются пробелы и табуляции;
- имя метки начинается после `:`;
- имя содержит от 1 до 31 непробельного символа;
- после имени метки допустимы только пробелы или табуляции;
- дополнительные аргументы запрещены;
- шестнадцатеричная, знаковая и вычисляемая форма адреса не поддерживается.

Примеры корректных строк:

```
trap 0 run :first
trap 104 run :classic_end
trap 111 run :expanded_end
```

Примеры некорректных строк:

```
TRAP 10 RUN :handler
trap -1 run :handler
trap 0x10 run :handler
trap 112 run :handler
trap 10 :handler
trap 10 run handler
trap 10 run :handler extra
```

Для двухбайтовой команды указывается адрес её первого байта - кода операции, а не адрес байта-операнда.

5. Адресное пространство

Парсер допускает весь диапазон программной памяти расширенного режима:

Режим	Допустимые адреса
Классический	0…104
Расширенный	0…111

Диапазон синтаксически проверяется при загрузке сценария, но соответствие фактическому размеру программной памяти проверяется перед каждым сценарным `run`.

Поэтому строка `trap 110 run :h` может быть корректно загружена, однако попытка `run` в классическом режиме завершится ошибкой: активная ловушка находится за пределами текущей программной памяти. В расширенном режиме тот же адрес допустим.

6. Две отдельные фазы: проверка и активация

Важно различать:

1. построение индекса ловушек при загрузке текущего текста;
2. активацию конкретной ловушки при исполнении её строки.

6.1. Проверка всего сценария

Функция `build_trap_index()` в `code/m61_text.cpp` сканирует весь текст сценария до исполнения первой команды. Для каждой строки, начинающейся с `trap`, она:

- разбирает синтаксис;
- проверяет диапазон адреса;
- запрещает повторное объявление того же адреса;
- находит целевую метку в заранее построенном индексе меток;
- записывает соответствие «адрес → номер метки» в таблицу `trap_targets`.

Следствия:

- ошибка обнаруживается даже в недостижимой ветви;
- метка может располагаться раньше или позже объявления;
- один обработчик может обслуживать несколько адресов;
- один адрес нельзя объявить дважды даже с одинаковой меткой;
- отсутствие целевой метки является ошибкой загрузки, а не поздней ошибкой срабатывания.

6.2. Активация строки

При фактическом исполнении корректной строки `trap` интерпретатор только устанавливает бит соответствующего адреса в `active_traps`.

Например:

```
run :skip
trap 10 run :never_activated
:skip
```

Объявление будет проверено, и метка `:never_activated` обязана существовать, но ловушка адреса `10` не станет активной, поскольку строка была пропущена.

6.3. Снятие ловушки

Отдельной команды `untrap` нет. Активная ловушка действует до одного из следующих событий:

- возврат из текущего вложенного сценария;
- загрузка другого корневого сценария;
- отмена M61 клавишей ESC;
- долгое нажатие `Cx`, снимающее одновременно все активные `trap` и `bind`;
- завершение M61 без оставшихся активных ловушек;
- ошибка, завершающая текущий M61 runner.

Обычный `ret` корневого сценария намеренно не завершает M61, если в текущем кадре осталась хотя бы одна активная ловушка. Подробности приведены в разделе 15.

Повторная загрузка того же корневого M61-файла сохраняет его уже активные ловушки. Поэтому `run`, расположенный до строк `trap`, при первом запуске выполняется без перехвата, а при повторном запуске того же файла использует ловушки, активированные ранее. Если файл между запусками был переписан, сохраняются только адреса, для которых в новой версии по-прежнему существует корректное объявление `trap`.

Если длинное `Cx` нажато в режиме `WATCH_EVENTS`, наблюдатель завершается и освобождает boundary hook и дисплей. Во время ещё выполняющегося сценария очищаются также сохранённые карты в кадрах возврата, поэтому старые ловушки и привязки не появятся снова после `ret`.

Штатный останов программы МК-61 сам по себе ничего не очищает. После него M61 продолжает сценарий со строки за `run`, а корневой `ret` сохраняет активные `trap` и `bind` в режиме `WATCH_EVENTS`.

7. Какие данные отслеживает M61

Основные структуры находятся в `code/m61_text.cpp`.

Поле	Назначение
`active_traps`	Битовая карта активированных адресов текущего сценария
`trap_targets`	Таблица «адрес → индекс метки обработчика»
`labels`	Индекс меток текущего сценария
`trap_pending`	Ядро уже попросило останов, но контекст ещё не сохранён
`pending_trap_address`	Адрес обнаруженной ловушки
`pending_trap_target`	Индекс выбранной метки
`trap_context_valid`	В `trap_context` находится сохранённый контекст
`trap_context`	Непрозрачный полный снимок состояния калькулятора

Поле	Назначение
`suspended_trap_address`	Адрес команды, перед которой остановлена машина
`bypass_trap_once`	Требуется одноразово пропустить ловушку после `ret`
`bypass_trap_address`	Адрес одноразового пропуска
`boundary_hook_installed`	Установлен ли наблюдатель границ команд

Для 112 адресов битовая карта занимает 14 байт. Таблица целей содержит по одной записи на каждый адрес. Неактивная или необъявленная цель обозначается внутренним значением `INVALID_TRAP_TARGET`.

В прошивках STM32F4 отдельные биты живой карты читаются и изменяются через SRAM bit-band alias Cortex-M4. Поэтому проверка на границе команды обращается к биту как к отдельному 32-битному слову, а установка и сброс выполняются атомарной записью без цикла «прочитать — изменить — записать». Это обычный C++-доступ через `volatile`-указатель, без вставок ассемблера. Настольные тесты используют эквивалентные сдвиг и маску.

Активная битовая карта входит в `ScriptFrame`, поэтому принадлежит конкретному кадру сценария. Таблицы целей и меток при восстановлении кадра перестраиваются по его исходному тексту.

8. Состояния исполнителя M61

Поле `RunnerState` принимает пять значений:

Состояние	Смысл
`IDLE`	Сценарий не исполняется
`EXECUTING`	M61 исполняет строки сценария
`WAIT_RUN_STOP`	M61 ждёт завершения программы, запущенной сценарным `run`
`WAIT_TIME`	Выполняется неблокирующий `wait`
`WATCH_EVENTS`	Сценарий дошёл до корневого `ret`, но продолжает следить за `trap` и `bind`

`trap_pending` и `trap_context_valid` - ортогональные признаки, а не отдельные значения `RunnerState`. Это позволяет точно различать:

- программу, остановленную ядром, но ещё не переданную обработчику;
- уже работающий обработчик с сохранённым контекстом;
- обычное ожидание останова программы;
- постоянное наблюдение за ручными запусками.

Функция `calculator_suspended()` возвращает истину как при `trap_pending`, так и при `trap_context_valid`. Благодаря этому никакой другой код не успевает изменить калькулятор в промежутке между остановом ядра и сохранением контекста.

9. Регистрация наблюдателя границ команд

Публичный интерфейс ядра объявлен в `code/mk61emu_core.h`:

```
struct Mk61ProgramBoundaryContext {
  u8 address;
  u8 opcode;
};

using Mk61ProgramBoundaryHook =
    bool (*)(const Mk61ProgramBoundaryContext& context,
             void* user_data);
```

Возвращаемое значение имеет следующий смысл:

- `false` - ядро продолжает исполнение команды;
- `true` - текущий вызов `core_61::step()` завершается до исполнения команды.

В ядре может быть зарегистрирован только один такой наблюдатель. Регистрация отклоняется, если:

- передан нулевой callback;
- наблюдатель уже зарегистрирован;
- производится попытка изменить регистрацию из самого callback.

M61 регистрирует наблюдатель при старте сценария. Если слот занят, сценарий завершается сообщением `calculator boundary hook is unavailable`.

При окончании, отмене или ошибке M61 снимает регистрацию. Указатель callback не входит в снимок калькулятора: жизненным циклом наблюдателя управляет M61.

10. Где находится граница команды в виртуальной машине

Механизм реализован в `code/mk61emu_core.cpp`.

10.1. Точка упреждающей выборки

Для исполняемой из программной памяти команды граница распознаётся в модели ИК1302 на микродресе ROM `0x06`. Это не адрес пользовательской программы, а внутренняя точка микропрограммы процессора.

К этому моменту:

- предыдущая пользовательская команда уже закончена;
- код следующей команды уже выбран в регистрах ИК1302;
- эмулируемый указатель программы ещё обозначает адрес этой команды;
- сама следующая команда ещё не начала изменять состояние калькулятора.

Ядро вычисляет:

```
address = get_IP() % program_steps
opcode  = low_nibble(R30) | high_nibble(R33)
```

Затем формируется `Mk61ProgramBoundaryContext`.

10.2. Порядок вызовов

На одной границе порядок принципиально следующий:

```
завершение предыдущей команды
        ↓
AFTER-фаза перехватчиков предыдущей команды
        ↓
наблюдатель границы программы
        ↓
если останов не запрошен - BEFORE-фаза перехватчиков новой команды
        ↓
исполнение новой команды
```

Следовательно, обработчик `trap` видит:

- все результаты предыдущей команды;
- исходный код операции новой команды;
- состояние до любого эффекта новой команды.

Если наблюдатель вернул `true`, командные перехватчики для новой команды в этом проходе ещё не вызываются.

10.3. Только программа, не клавиатура

Клавиатурный путь ИК1302 использует другую внутреннюю точку (`0x97`) и не создаёт событие программной границы. Поэтому `trap` отслеживает команды, прочитанные из программной памяти, а не такие же коды, введённые нажатием клавиш.

11. Двухбайтовые команды и исключение операндов

Ловушка всегда относится к коду операции. Байты адресных операндов не должны выглядеть как самостоятельные команды.

Для обычных двухбайтовых переходов это обеспечивается самим ходом микропрограммы и положением IP. У команды вызова подпрограммы (CALL, код `0x53`) есть особенность: её байт-операнд проходит общую точку упреждающей выборки два раза.

Чтобы не создавать ложные границы, ядро хранит отдельный LIFO:

- адрес ожидаемого операнда CALL;
- число его оставшихся наблюдений;
- глубину вложенности.

Для каждого выполненного CALL записывается адрес `(program_address + 1) % program_steps` и два ожидаемых посещения. Оба посещения поглощаются и не передаются M61. Вложенные вызовы поддерживаются LIFO глубиной 5, соответствующим моделируемой глубине вызовов.

Тесты проверяют:

- переход с двухбайтовым адресом;
- одиночный CALL;
- возврат из подпрограммы;
- вложенные CALL;
- отсутствие ложного срабатывания на байтах-операндах.

12. Полный путь одного срабатывания

Срабатывание проходит по следующей цепочке:

```
ИК1302, ROM 0x06
    ↓
ядро декодирует адрес и opcode
    ↓
Mk61ProgramBoundaryHook
    ↓
m61_text::program_boundary_hook()
    ↓
проверка active_traps[address]
    ↓
trap_pending = true
    ↓
callback возвращает true
    ↓
core_61::step() завершается до команды
    ↓
пакет режима MAXIMUM немедленно прекращается
    ↓
следующий проход основного цикла вызывает m61_text::service()
    ↓
создаётся кадр возврата TRAP
    ↓
core_61::save_context()
    ↓
переход M61 к метке обработчика
```

12.1. Callback намеренно минимален

`program_boundary_hook()` не сохраняет большой контекст и не исполняет сценарий внутри микрокода. Он только:

1. убеждается, что M61 находится в `WAIT_RUN_STOP` или `WATCH_EVENTS`;
2. обрабатывает одноразовый пропуск после `ret`;
3. проверяет бит адреса;
4. проверяет существование цели;
5. записывает адрес и цель в поля `pending_trap_*`;
6. возвращает `true`.

Так callback остаётся коротким, не реентерабельным и не выполняет тяжёлую работу внутри шага виртуальной машины.

12.2. Немедленный выход из пакетного исполнения MAXIMUM

Ядро выставляет признак `mk61_program_boundary_yielded`. Планировщик в `code/automate.hpp` проверяет его после каждого шага и немедленно покидает текущий пакет MAXIMUM.

Это важно: даже при высокой скорости между обнаружением ловушки и `m61_text::service()` не исполняется следующая команда программы.

12.3. Передача обработчику

На следующем обслуживании M61:

1. сохраняет текущий кадр сценария в `ReturnFrame` вида `TRAP`;
2. вызывает `core_61::save_context()`;
3. отмечает `trap_context_valid`;
4. запоминает адрес останова;
5. переводит позицию сценария к целевой метке;
6. переходит в `EXECUTING`.

Сохранение выполняется до первой команды обработчика.

13. Что сохраняется в контексте калькулятора

Публичный `core_61::ContextBuffer` является непрозрачным буфером размером 1280 байт с выравниванием 8 байт. Внутренняя структура `CoreContextSnapshot` содержит:

- сигнатуру формата `MK61`;
- всю кольцевую память `ringM`;
- полные структуры ИК1302, ИК1303 и ИК1306;
- состояние оболочки `MK61Emu`;
- регистры, стек, указатель программы и внутреннее микросостояние;
- защёлку дисплея и состояние ошибки;
- позицию запятой для шага назад;
- признак режима редактирования;
- классический или расширенный режим;
- состояние улучшенного генератора случайных чисел;
- активную пользовательскую команду и последовательность команд;
- состояние завершения клавиатурной команды;

- LIFO адресов операндов CALL и счётчики их посещений.

Перед записью публичный буфер обнуляется. При восстановлении проверяется сигнатура, а некорректная глубина внутреннего LIFO сбрасывается.

Снимок не содержит:

- состояние самого интерпретатора M61;
- таблицы и битовые карты ловушек;
- указатель зарегистрированного boundary hook;
- внешнюю очередь терминала;
- физическое состояние кнопок.

Эти данные принадлежат вызывающему коду и живут отдельно.

`ContextBuffer` предназначен для сохранения и восстановления в рамках того же запуска той же прошивки. Внутренние структуры микросхем содержат процессные ссылки, поэтому буфер не является переносимым форматом файла или сериализацией для другой версии прошивки.

14. Возврат из обработчика

Обработчик обязан закончиться явной командой:

```
ret
```

Для кадра вида TRAP она выполняет:

1. `core_61::restore_context(trap_context)`;
2. снимает признак `trap_context_valid`;
3. включает `bypass_trap_once`;
4. записывает адрес останова в `bypass_trap_address`;
5. восстанавливает кадр сценария;
6. возвращает предыдущее состояние `WAIT_RUN_STOP` или `WATCH_EVENTS`.

Если обработчик доходит до конца файла без `ret`, M61 выдаёт:

```
trap handler reached end of script without ret
```

Такое завершение намеренно считается ошибкой: неявно угадать требуемый тип возврата и продолжение программы небезопасно.

14.1. Почему нужен одноразовый пропуск

Сохранённый контекст указывает на ту же ещё не исполненную команду. Сразу после восстановления виртуальная машина снова приходит на ту же границу. Если просто восстановить контекст, ловушка сработает бесконечно.

Поэтому `program_boundary_hook()` проверяет одноразовый пропуск:

- если текущий адрес совпадает с `bypass_trap_address`, callback возвращает `false`;
- флаг пропуска сразу погашается;
- команда исполняется обычным образом;
- при следующем приходе на этот адрес активная ловушка снова действует.

Это не временное удаление бита и не отключение всех ловушек. Пропускается только непосредственное повторное предъявление сохранённой границы.

14.2. Выбор углового режима

Полный снимок содержит угловой режим, но клавиши P, Г и ГРД разрешены во время обработчика как внешний выбор пользователя.

Перед восстановлением M61 запоминает текущий внешний селектор, восстанавливает весь контекст, а затем повторно применяет выбранный селектор. Поэтому переключение углового режима во время обработчика не теряется после `ret`.

Остальное состояние калькулятора восстанавливается точно из снимка.

15. Возврат после сценарного и ручного запуска

Ловушка может сработать в двух режимах.

15.1. Сценарный `run`

Перед запуском M61:

- проверяет, что каждая активная ловушка входит в текущую программную память;
- переводится в `WAIT_RUN_STOP`;
- запускает программу калькулятора.

Если ловушка сработала, кадр `TRAP` запоминает именно `WAIT_RUN_STOP`. После `ret` программа продолжает исполнение. Когда она обычно остановится, M61 продолжит сценарий со строки после `run`.

15.2. Постоянное наблюдение `WATCH_EVENTS`

Если корневой сценарий выполняет `ret`, но у него остаётся хотя бы одна активная ловушка `trap` или привязка `bind`, M61 не отключается. Он переходит в `WATCH_EVENTS`:

```
hin
...
trap 10 run :frame
ret

:frame
print "TRAP HIT"
ret
```

После корневого `ret` пользователь может вручную нажимать B/O, задавать адрес и запускать программу клавишей С/П. Ловушки продолжат действовать.

При попадании кадр `TRAP` запоминает `WATCH_EVENTS`, а обработчик после `ret` возвращает M61 к постоянному наблюдению. Обычный ручной останов программы не завершает наблюдение.

Чтобы выйти из этого режима и снять ловушки, нужно нажать `ESC` либо загрузить другой корневой сценарий.

16. Обработчики, метки и вложенные сценарии

Стек возвратов M61 содержит кадры трёх видов:

Вид	Назначение
`SCRIPT`	Возврат из вложенного файла M61
`LABEL`	Локальный вызов `run :label` из обработчика
`TRAP`	Возврат к остановленному калькулятору

16.1. `run :label` вне обработчика

В обычном сценарии `run :label` является переходом к метке. Кадр возврата не создаётся, поэтому такой переход можно использовать для циклов.

16.2. `run :label` внутри обработчика

При действующем `trap_context_valid` та же команда становится локальным вызовом:

```
:frame
run :common
print "HANDLER CONTINUES"
ret

:common
stk
ret
```

Первый `ret` в `:common` возвращает в `:frame`; второй восстанавливает калькулятор. Это позволяет выносить общую диагностику в подпрограммы M61.

16.3. Активные ловушки вложенных файлов

Каждый `ScriptFrame` хранит собственную битовую карту активных ловушек. Новый вложенный сценарий начинает с пустой карты и строит собственные индексы меток и целей. При возврате:

- его ловушки отбрасываются;
- карта родителя восстанавливается;
- индексы родителя заново строятся по сохранённому исходному тексту.

Таким образом, ловушки дочернего файла не протекают в родительский, а ловушки родителя не становятся активными внутри дочернего кадра.

Открытие другого файла из активного обработчика запрещено правилами безопасности терминала.

16.4. Глубина стека

Глубина вложенности файлов ограничена 8. Общий стек `ReturnFrame` имеет 9 элементов: дополнительный элемент позволяет создать кадр TRAP на максимальной глубине вложенности файлов.

Кадры LABEL используют тот же общий стек. Поэтому глубоко вложенный обработчик может исчерпать его раньше. Соответствующие ошибки:

```
trap return stack is full
label return stack is full
```

17. Разрешённые команды обработчика

Пока сохранён контекст ловушки, нельзя допустить изменение калькулятора в обход снимка. Фильтр `terminal_command_allowed_in_trap()` разрешает только следующие команды:

```
ver
list
dump
pub
lasm
isa
hout
reg
1302
ring
stk
led
beep
if
run :label
print
wait
ret
```

Группы назначения:

- `ver` - информация о версии;
- `list`, `dump`, `pub`, `lasm`, `isa`, `hout` - чтение и представление программы;
- `reg`, `1302`, `ring`, `stk` - чтение состояния;
- `led`, `beep`, `print` - внешняя индикация;

- `wait` - неблокирующая задержка;
- `if` - условное исполнение, причём вложенная команда проходит тот же фильтр;
- `run :label` - локальный вызов обработчика;
- `ret` - возврат.

Для `run` разрешена только точная форма перехода к локальной метке. Обычный запуск программы и открытие файла запрещены.

В частности, обработчик не может:

- вводить или исполнять команды калькулятора;
- посылать клавиши;
- менять регистры;
- выполнять `poke`;
- загружать другую программу;
- запускать ассемблерную запись;
- менять системные параметры, способные затронуть ядро.

Интерактивный терминал использует тот же режим фильтрации, когда `calculator_suspended()` истинно. Поэтому обойти ограничения обработчика, набрав опасную команду вручную, нельзя.

Если команда не разрешена, выводится:

```
Command is not allowed in an M61 trap handler: <command>
```

А сценарий M61 получает ошибку:

```
terminal command failed
```

18. Клавиатура и основной цикл прошивки

Основной цикл в `code/mk61s-M.ino` обслуживает M61 до очередного обычного шага калькулятора. Пока `calculator_suspended()` истинно:

- обычные события клавиатуры калькулятора не передаются ядру;
- накопленные обычные события отбрасываются;
- `mk61_baseloop_hook()` не выполняет фоновое изменение калькулятора;
- ESC остаётся доступным для отмены;
- Р, Г и ГРД остаются доступными для внешнего выбора углового режима.

Специальные кнопки проверяются и во время `wait` внутри обработчика, поэтому задержка не лишает пользователя возможности отменить сценарий.

Короткие нажатия ESC и угловых клавиш обрабатываются немедленно и не теряются из-за очистки обычной очереди.

19. `wait`, вывод и владение дисплеем

`wait` внутри обработчика переводит M61 в `WAIT_TIME`, но не восстанавливает калькулятор. Контекст остаётся сохранённым до истечения задержки и последующего `ret`.

Первый вывод M61 может передать дисплей во владение сценария. Пока сценарий владеет дисплеем, обычные обновления калькулятора подавляются, чтобы диагностический кадр не был немедленно затёрт.

При окончательном завершении или отмене M61 владение освобождается, а дисплей калькулятора перерисовывается.

В режиме WATCH_EVENTS сценарий остаётся активным, поэтому ранее захваченное владение дисплеем также может сохраняться до ESC или загрузки другого сценария.

20. Отмена, ошибки и очистка

Центральная очистка выполняется через `stop_runner()` и `clear_trap_runtime()`.

Если снимок уже существует, очистка сначала пытается восстановить контекст. Затем она:

- сбрасывает `trap_pending`;
- сбрасывает признаки сохранённого контекста;
- сбрасывает одноразовый пропуск;
- снимает boundary hook;
- очищает стек возвратов и кадры сценариев;
- переводит M61 в `IDLE`;
- освобождает дисплей.

Клавиша ESC дополнительно посылает останов калькулятору, если после восстановления он находится в состоянии `RUN`, очищает события ввода и перерисовывает дисплей.

При внутренней ошибке M61 прекращает runner, восстанавливает сохранённый контекст и снимает наблюдатель. После этого калькулятор снова управляется обычным циклом прошивки. Если контекст ещё не был сохранён, восстанавливать нечего: ядро уже находится на безопасной границе перед командой.

21. Диагностические сообщения

21.1. Ошибки загрузки и индексации

Сообщение	Причина
`invalid trap (use: trap <0..111> run :label)`	Неверный синтаксис или адрес вне 0…111
`duplicate trap address`	Один адрес объявлен более одного раза
`trap label not found`	Целевая метка отсутствует
`duplicate label`	Повторно объявлена метка
`too many labels (maximum 64)`	Превышен предел индекса меток
`invalid label (use 1..31 non-space characters)`	Некорректная строка метки

21.2. Ошибки исполнения

Сообщение	Причина
`active trap address is outside current program memory`	Адрес активной ловушки не существует в текущем режиме
`calculator boundary hook is unavailable`	Единственный слот наблюдателя занят
`trap return stack is full`	Нельзя создать кадр возврата ловушки
`label return stack is full`	Нельзя создать локальный вызов метки
`cannot save calculator trap context`	Ядро отклонило сохранение снимка
`cannot enter trap handler`	Не удалось подготовить кадр обработчика
`cannot restore calculator trap context`	Ядро отклонило восстановление снимка
`trap handler reached end of script without ret`	Обработчик завершился неявно
`terminal command failed`	Команда обработчика запрещена или завершилась ошибкой

В диагностике сценария сохраняются имя файла и номер текущей строки.

22. Пределы реализации

Ресурс	Предел
Адреса ловушек	112 (0…111)
Размер битовой карты	14 байт
Метки одного сценария	64
Длина имени метки	31 символ
Вложенность файлов	8
Общий стек возвратов	9 кадров
LIFO операндов `CALL` в ядре	5
Бюджет строк M61 за одно обслуживание	8
Буфер снимка ядра	1280 байт

Дополнительные ограничения:

- один адрес можно связать только с одной меткой;
- несколько адресов можно связать с одной меткой;
- отдельного отключения одной ловушки нет;
- обработчик не получает адрес срабатывания как переменную M61;
- повторная калькуляторная ловушка внутри обработчика невозможна, потому что виртуальная машина заморожена;
- локальные вызовы меток внутри обработчика возможны;
- одновременно существует только один программный boundary hook;
- снимок контекста не переносим между запусками или версиями прошивки;
- фактическое число объявлений также ограничивается размером файла сценария.

Если нескольким адресам назначен один обработчик, он может анализировать сохранённое состояние калькулятора, но прямого встроенного параметра «сработавший адрес» язык сейчас не предоставляет.

23. Примеры

23.1. Несколько адресов с одним обработчиком

```
trap 10 run :trace
trap 20 run :trace
trap 30 run :trace
run
ret

:trace
reg
stk
ret
```

Каждый адрес имеет отдельный активный бит, но одна запись метки может быть общей.

23.2. Ловушка, активируемая позднее

```
run
trap 10 run :trace
run
ret

:trace
print "TRAP IS ACTIVE ON THE SECOND RUN"
ret
```

Во время первого `run` строка `trap` ещё не исполнена. Во время второго запуска ловушка уже активна.

23.3. Общая подпрограмма обработчиков

```
trap 10 run :first
trap 20 run :second
run
ret

:first
print "HANDLER 10"
run :common
ret

:second
print "HANDLER 20"
run :common
ret

:common
stk
ret
```

`ret` из `:common` возвращает в вызывающий обработчик, а следующий `ret` возвращает к калькулятору.

23.4. Наблюдение за ручными запусками

```
trap 10 run :trace
ret

:trace
print "STEP 10"
ret
```

После первого `ret` M61 остаётся в `WATCH_EVENTS`. Сценарий завершится только после ESC, загрузки другого сценария или ошибки.

23.5. Задержка диагностического кадра

```
:trace
print "BEFORE COMMAND"
stk
wait 1000
ret
```

Калькулятор остаётся точно в сохранённом состоянии на протяжении всей секунды. После `ret` состояние восстанавливается, и перехваченная команда исполняется.

24. Отличие от других перехватчиков

В проекте есть четыре разных уровня расширения:

Механизм	Ключ	Момент	Может изменить opcode	Привязан к адресу программы
`trap` M61	Адрес	До команды программы	Нет	Да
`bind` M61	Код операции с клавиатуры	BEFORE/AFTER команды	Да, заменяет на NOP	Нет
Command hook	Код операции	BEFORE/AFTER команды	Да	Нет
ROM hook	Микроадрес микросхемы	Внутри микрокода	Да, на уровне микрооперации	Нет

Boundary hook, используемый `trap`, специально отделён от command hooks:

- он видит адрес программной памяти;
- он может попросить `step()` уступить управление;
- он срабатывает до BEFORE-фазы командного перехватчика;
- он не изменяет код программы или opcode.

Форма `bind <opcode>`, напротив, построена на паре command hooks для каждого opcode: BEFORE поглощает клавиатурную операцию, а AFTER передаёт управление M61 только после завершения нейтральной замены. Команды, прочитанные из программы, он игнорирует.

bind key **<**код**>** down/up run :label запускается диспетчером физических событий до штатной обработки. Во время RUN запрос ждёт границы команды, а во время `trap` — восстановления снимка. Обработчики сериализованы; изменения регистров сохраняются. OK имеет код 13; отдельная форма `bind OK` удалена. Полная таблица кодов и правила очереди приведены в руководстве M61.

Подробности других механизмов приведены в документах «Перехватчики команд» (`MK61s-mini-Command-Hooks.md`) и «Перехватчики ROM» (`MK61s-mini-ROM-Hooks.md`).

25. Карта исходного кода

Файл	Ответственность
`code/m61_text.cpp`	Парсер `trap`, индексы, битовые карты, состояния M61, callback, вход в обработчик, `ret`, очистка
`code/m61_text.hpp`	Публичное состояние и обслуживание M61
`code/mk61emu_core.h`	API boundary hook и непрозрачного контекста
`code/mk61emu_core.cpp`	Граница ИК1302, останов шага, фильтрация операндов, снимок ядра
`code/automate.hpp`	Немедленное завершение пакета MAXIMUM после yield
`code/mk61s-M.ino`	Порядок основного цикла, клавиатура, ESC, угловые клавиши
`code/terminal.hpp`	Разбор команд и применение режима безопасности
`code/terminal_command_ids.hpp`	Белый список команд обработчика
`tests/m61_text_self_test.cpp`	Семантика M61, срабатывание, возврат, повтор, ошибки
`tests/mk_math_self_test.cpp`	Граница ядра, yield, снимок, двухбайтовые команды и CALL
`tests/terminal_self_test.cpp`	Белый список и запрет опасных команд

26. Тестируемые инварианты

Автоматические тесты закрепляют следующие свойства:

- callback вызывается перед исполнением команды;
- запрос останова завершает текущий `step()`;
- следующий шаг снова видит ту же границу, если владелец не разрешил проход;
- снимок восстанавливает точный IP и состояние ядра;
- операнды переходов и CALL не считаются командами;
- предыдущая команда полностью завершена до обработчика;
- перехваченная команда не выполнена до `ret`;
- после `ret` она проходит один раз;
- следующий приход на адрес снова вызывает обработчик;
- ловушка сохраняется для последующих ручных запусков;
- при первом запуске строка `trap` действует только после фактического исполнения; повторный запуск того же корневого файла сохраняет ранее активированные ловушки;
- неверные, дублирующиеся и отсутствующие цели отвергаются;
- адреса 105…111 зависят от расширенного режима;
- конец обработчика без `ret` является ошибкой;
- при ошибке сохранённый контекст восстанавливается, а hook снимается;
- `wait` удерживает снимок до продолжения;
- смена углового режима переживает `ret`;
- `run :label` внутри обработчика создаёт локальный возврат;
- терминал не позволяет изменить калькулятор во время остановки.

Основные команды локальной проверки:

```
tests/run_m61_text_tests.sh
tests/run_mk_math_tests.sh
tests/run_terminal_tests.sh
```

27. Требования при изменении механизма

При развитии `trap` необходимо сохранять следующие инварианты:

- callback границы не должен исполнять сценарий или выполнять длительную работу;
- после `yield` планировщик MAXIMUM обязан выйти до следующего шага ядра;
- `calculator_suspended()` должен быть истинным уже для `trap_pending`;
- любые новые команды белого списка не должны мутировать сохранённое состояние калькулятора;
- команды в хвосте `if` обязаны проходить тот же фильтр;
- любое новое состояние ядра, влияющее на дальнейшее исполнение, должно войти в `CoreContextSnapshot`;
- восстановление должно оставлять возможность ровно одного прохода перехваченной команды;
- логика двухбайтовых команд обязана исключать операнды на всех ветвях и при вложенных вызовах;
- ошибка и отмена должны снимать singleton hook;
- активные карты должны оставаться изолированными между кадрами сценариев.

Если добавляется команда снятия ловушки, перед реализацией требуется определить её поведение отдельно для текущего кадра, родительских кадров, `WAIT_RUN_STOP`, активного обработчика и `WATCH_EVENTS`.

28. Итоговая модель

Механизм можно свести к шести независимым обязанностям:

1. M61 индексирует все объявления и активирует адреса только при исполнении соответствующих строк.
2. Ядро распознаёт точную границу команды программы внутри ИК1302 и исключает байты-операнды.
3. Boundary hook просит yield, не исполняя тяжёлый код внутри виртуальной машины.
4. Основной цикл передаёт управление M61, который сохраняет полный снимок и запускает метку обработчика.
5. Белый список и блокировка ввода не позволяют изменить замороженный калькулятор в обход снимка.
6. Возврат восстанавливает состояние: `ret` даёт той же команде один проход, сохраняя ловушку для будущих попаданий.

Именно сочетание этих уровней обеспечивает главное обещание `trap`: обработчик наблюдает полностью завершённое состояние перед выбранной командой, а после возврата программа продолжается так, как если бы диагностический сценарий был вставлен между двумя командами и не изменил калькулятор.